

Project Context: AI Course Intelligence / Semester Planning Platform

I am building a serious production-oriented AI project intended to have real users, not just be a demo. The goal is to showcase strong skills in multi-agent systems, RAG, backend development, databases, structured LLM outputs, web research, frontend visualization, deployment, and product engineering.

Core Product Idea

The product is an AI-powered semester/course intelligence platform for university students.

A student creates a semester and uploads syllabi for approximately 4–6 courses.

The system analyzes all courses and produces a visual dashboard showing:

- Overall course difficulty ranking, potentially S–D tiers
- Expected workload per course
- Recommended weekly time dedication
- Continuous-study requirement / "cram-friendliness"
- Conceptual difficulty
- Exam pressure
- Project/assignment workload
- Deadline density
- Confidence score
- Evidence explaining each score
- Combined calendar of exams, assignments, projects, and deadlines
- Weekly semester workload represented visually using colors such as green/yellow/red
- Priority ranking that can change depending on upcoming deadlines

The system should answer questions such as:

- Which course requires the most consistent effort?
- Which courses can be studied closer to an exam?
- Which upcoming week will be especially difficult?
- Which course should I prioritize this week?
- Why was a course ranked S-tier?
- What evidence supports the workload estimate?

However, the product should NOT primarily be a chatbot.

UX Philosophy

The LLM should act as an invisible reasoning engine in the backend.

The website should be visualization-oriented rather than chat-oriented.

The ideal user experience is:

1. Student opens the dashboard.
2. Within seconds they can visually understand:
 - hardest courses
 - current priorities
 - upcoming stressful weeks
 - deadlines
 - workload distribution
3. If they want additional explanation, they can click "Why?" on a course.
4. A drawer/modal expands with reasoning, evidence, confidence, and source links.

The user should not need to repeatedly converse with an LLM.

This is intentionally different from the typical "Chat with your documents" application.

Main Data Sources

The AI should combine three major information sources.

1. Uploaded Syllabi

The system extracts structured data including:

- course code
- professor
- semester
- assignments
- project information
- labs
- quizzes
- midterms
- final exam
- weights
- due dates
- attendance requirements
- grading rules
- late policies

- office hours where relevant

Important principle:

Structured syllabus information should be stored as structured database records, NOT only as embeddings.

For example:

- Exam date -> PostgreSQL
- Assignment weight -> PostgreSQL
- Course professor -> PostgreSQL

The original syllabus text/chunks can additionally be embedded for RAG.

2. External Web Research

A research agent should search for course-specific external evidence.

Potential sources include:

- Reddit, especially university-specific subreddits
- university/course pages
- professor/course discussions
- public course reviews
- potentially professor-review sites where permitted

Example searches:

- site.com/r/uAlberta "CMPUT 301"
- site.com/r/uAlberta "CMPUT 301" "Professor Name"
- "CMPUT 301 workload"
- "CMPUT 301 project difficult"

External reviews should be considered noisy evidence, NOT ground truth.

The system should track:

- source URL
- source type
- publication date
- professor/course relevance
- extracted claim
- confidence/relevance
- potentially student sentiment

It should recognize disagreement such as:

"easiest course ever"

versus

"this course destroyed me"

rather than blindly treating every review equally.

3. LLM Reasoning

The LLM synthesizes:

- syllabus structure
- external student experiences
- professor-specific information
- assessment distribution
- course structure
- deadline pressure

The LLM estimates fuzzy properties such as workload or continuous-study requirements.

Normal code should handle deterministic operations such as sorting, dates, thresholds, calendar calculations, authentication, database writes, etc.

Guiding principle:

"LLMs should reason; software should enforce."

Backend / Frontend Contract

The backend should return structured JSON.

The frontend should NOT parse random natural-language LLM responses.

Example conceptual API response:

```
{  
  "semester": "Fall 2026",  
  "courses": [  
    {  
      "course_code": "CMPUT 301",
```

```

"overall_rank": "S",
"difficulty_score": 8.4,
"weekly_hours": {
  "min": 7,
  "max": 10
},
"continuous_study_score": 9.1,
"confidence": 0.87,
"reasoning": {
  "summary": "High continuous workload because of weekly labs and a cumulative team project.",
  "factors": [
    "Weekly labs",
    "Four major assignments",
    "Project worth 35%",
    "Student reviews repeatedly mention project workload"
  ]
},
"evidence": []
},
],
"calendar": [
  {
    "week": "2026-10-12",
    "workload_score": 9.2,
    "level": "critical",
    "events": []
  }
]
}

```

React simply renders this structured application state.

For example:

```

overall_rank == "S"
-> render S-tier badge

```

```

workload_score >= threshold
-> render red workload week

```

```

confidence == 0.87
-> display 87%

```

```

reasoning.factors
-> populate expandable explanation panel

```

Multi-Agent Architecture

The project should use multiple specialized agents only where specialization actually helps.

Do NOT create artificial agents just so the project can say "multi-agent."

The intended design is specialized agents + orchestration + structured state.

Potential Agents

Syllabus Intelligence Agent

Responsibilities:

- understand syllabus PDFs
- extract structured course information
- identify assessments, weights, deadlines, professor, etc.

It should return strict structured output.

Example:

```
{  
"course": "CMPUT 301",  
"professor": "...",  
"assignments": 4,  
"labs": 8,  
"midterms": 1,  
"project_weight": 35,  
"assessment_dates": []  
}
```

Research Agent

Responsibilities:

- search external sources
- search Reddit/course-specific discussions
- search professor-specific information
- extract relevant evidence
- distinguish fact from subjective student opinion

- return citations

Possible tools:

- web_search()
- reddit_search()
- fetch_page()
- extract_relevant_comments()

Workload / Difficulty Evaluator Agent

Receives:

- structured syllabus facts
- external research evidence
- course characteristics

Produces dimensions such as:

- conceptual difficulty
- weekly workload
- exam pressure
- project workload
- continuous-study requirement
- deadline density
- confidence

It should NOT necessarily directly decide the final UI ranking.

A deterministic backend formula can convert dimensions into the final priority score.

Example:

```
priority =
0.25 * workload
+ 0.20 * conceptual_difficulty
+ 0.20 * deadline_pressure
+ 0.15 * assessment_weighting
+ 0.10 * continuous_study
+ 0.10 * student_review_difficulty
```

Then code translates score ranges into S/A/B/C/D.

Critic / Reviewer Agent

This is preferred over fake "agent debates."

The critic receives the evaluator's claims and evidence and checks:

- Is there enough evidence?
- Are sources current?
- Are reviews for the correct professor?
- Are claims supported?
- Are there contradictory sources?
- Is the model over-weighting a small number of reviews?
- Is the confidence too high?

Possible result:

```
{  
"status": "revise",  
"problems": [  
  "Only one review refers to the current professor",  
  "Two sources are outdated"  
],  
"requested_research": [  
  "Find recent professor-specific evidence"  
]  
}
```

If revision is needed:

Critic

-> Research Agent

-> Evaluator

-> Critic again

There must be a maximum retry/revision count, e.g. 2.

If evidence remains weak, return low confidence rather than inventing certainty.

Planner Agent

Combines:

- assessments
- due dates
- expected preparation workload
- course priority
- student's available study time

Produces semester/week planning recommendations.

Eventually this can become a scheduling/optimization problem where deterministic algorithms and AI reasoning are combined.

What "Multi-Agent" Means Technically

An agent is conceptually:

LLM

- fixed system instructions
- structured state/context
- available tools
- ability to select actions

At the lowest level, agents are API calls.

For example:

```
response = llm.generate(  
    system_prompt=SYLLABUS_AGENT_PROMPT,  
    input=syllabus_text  
)
```

Then another specialized call:

```
research = llm.generate(  
    system_prompt=RESEARCH_AGENT_PROMPT,  
    input={  
        "course": "CMPUT 301",  
        "professor": "..."  
    },  
    tools=[web_search]  
)
```

The interesting engineering is not the API request itself.

The interesting engineering is:

- which agent runs
- what state it receives
- what tools it can call
- what structured schema it returns
- which agent runs next

- conditional branches
- retries
- confidence
- parallel execution
- failure handling

That is orchestration.

Orchestrator

The orchestrator does not necessarily need to be another LLM.

Normal Python/backend code can orchestrate the workflow.

Conceptual example:

```
async def analyze_course(course):

    syllabus = await syllabus_agent(course)

    research = await research_agent(
        course_code=syllabus.course_code,
        professor=syllabus.professor
    )

    evaluation = await workload_agent(
        syllabus=syllabus,
        research=research
    )

    critique = await critic_agent(
        evaluation=evaluation,
        evidence=research
    )

    if critique.status == "revise":

        additional_research = await research_agent(
            queries=critique.requested_research
        )

        evaluation = await workload_agent(
```

```
    syllabus=syllabus,  
    research=research + additional_research  
)
```

```
return evaluation
```

This already counts as a multi-agent workflow.

The architecture can be implemented manually initially and later represented using a graph-oriented orchestration framework such as LangGraph.

Hybrid Agentic Design

The overall application should be mostly deterministic.

Example:

Parse

- > Research
- > Evaluate
- > Verify
- > Rank
- > Plan

But individual agents can behave agentially.

For example, the Research Agent may perform:

Search

- > inspect results
- > decide evidence is insufficient
- > formulate another search
- > inspect again
- > stop when sufficient evidence exists

This gives us a hybrid system:

Deterministic workflow + agentic tool use inside selected nodes.

Fan-Out / Fan-In Architecture

When the user has 5 courses, they should not necessarily be analyzed serially.

Instead:

Semester

|

+-- Course Worker: CMPUT 201

+-- Course Worker: CMPUT 301

+-- Course Worker: BIOL 207

+-- Course Worker: STAT 252

+-- Course Worker: Course 5

These course analyses run concurrently.

Then all results are combined by a semester-level analyzer/ranker.

Conceptually:

```
results = await asyncio.gather(  
    analyze_course(course1),  
    analyze_course(course2),  
    analyze_course(course3),  
    analyze_course(course4),  
    analyze_course(course5)  
)
```

Then:

```
semester_analysis = analyze_semester(results)
```

This is a fan-out / fan-in architecture.

\

RAG Architecture

There are two conceptually separate retrieval systems.

Internal RAG

Student-owned files such as:

- syllabus PDFs
- course documents

Pipeline:

PDF

- > parser
- > cleaned text
- > chunks
- > embeddings
- > vector database

Used for things such as:

"When is Assignment 2 due?"

"What chapters are covered on the midterm?"

External Retrieval

Internet sources:

search query

- > web/search API
- > Reddit/university pages/etc.
- > relevant-content extraction
- > relevance scoring/filtering
- > evidence records
- > evaluator

Internal RAG and external web retrieval should remain distinguishable because they have different reliability and provenance.

Database Design

Use PostgreSQL.

Potential tables/entities:

- users
- semesters
- courses
- course_enrollments
- syllabi
- documents
- document_chunks
- assessments
- professors
- evidence
- external_sources
- course_evaluations
- study_plans
- workload_weeks

Use pgvector for embeddings instead of adding a completely separate vector DB initially.

Structured facts remain in SQL.

Unstructured semantic retrieval uses pgvector.

Recommended Technology Stack

Frontend:

- Next.js
- React
- TypeScript
- Tailwind CSS
- component library as needed
- charts/calendar visualization library

Backend:

- Python
- FastAPI
- Pydantic
- SQLAlchemy

Database:

- PostgreSQL
- **pgvector**

Files:

- S3-compatible object storage for syllabus PDFs

AI:

- LLM API with tool calling
- structured outputs
- embedding model

Agent orchestration:

- manual Python orchestration initially
- LangGraph or similar graph-based framework once workflow complexity justifies it

External research:

- Web search API
- Reddit API/search where permitted
- public university sources
- other sources only where terms permit automated access

Async/background processing later:

- Redis
- Celery/RQ/other worker system if needed

Deployment:

- Vercel for frontend
- suitable cloud backend host
- managed PostgreSQL
- object storage

Engineering:

- GitHub Actions
- Pytest
- Vitest
- logging
- Sentry-like error tracking
- agent tracing/observability
- analytics

Architecture style:

Use a modular monolith initially.

Do NOT prematurely create many microservices.

Possible Backend Folder Structure

backend/

api/

auth

courses

syllabi

semester

analysis

services/

pdf_service

search_service

embedding_service

ranking_service

agents/

syllabus_agent

research_agent

evaluator_agent

critic_agent

planner_agent

prompts/

syllabus.md

research.md

evaluator.md

critic.md
planner.md

orchestration/
course_workflow
semester_graph

models/
course
syllabus
assessment
evidence
evaluation

database/
...

Agents are only one part of the backend.

Ranking Philosophy

Do not simply ask:

"Rank these courses S–D."

Instead:

1. Extract objective facts.
2. Retrieve external evidence.
3. Estimate multiple meaningful dimensions.
4. Verify those estimates.
5. Compute a reproducible ranking algorithmically.

Possible dimensions:

- workload
- conceptual difficulty
- assessment density
- upcoming deadline pressure
- exam pressure
- project workload

- continuous-study requirement
- external student-review difficulty

The frontend can display the final tier.

Continuous Study Requirement

Avoid framing everything as:

"Can I cram this course?"

Internally use a more defensible metric:

Continuous Study Requirement

Example:

CMPUT 301

Continuous Study Requirement: 9.1/10

Reasons:

- weekly labs
- progressive project
- assignments build on previous work

A different course might have:

Continuous Study Requirement: 3.2/10

Reasons:

- few recurring deliverables
- grade dominated by exams
- modules relatively independent

The UI could still show a user-friendly label such as:

Cram-friendly: Low / Medium / High

Calendar / Workload Visualization

One of the main product features should be a semester workload heatmap.

The backend combines assessments across all courses and produces weekly workload levels.

Example:

Week 1 -> green

Week 2 -> green

Week 3 -> yellow

Week 4 -> red

The user should immediately see stressful periods.

This may incorporate:

- assessment weights
- assignment/project size
- exam dates
- estimated preparation hours
- overlapping deadlines
- course difficulty
- user-specific availability

The frontend should visually "hit the user in the face" with this information rather than hiding everything behind chat.

Important Product Principles

1. The website is an AI-powered application, not an AI chatbot.
2. The LLM lives primarily in the backend.
3. Backend agents return structured JSON.
4. The frontend renders visual state.
5. Users can inspect reasoning only when they want it.
6. Every important AI conclusion should expose supporting evidence.
7. Confidence should be explicit.
8. Weak evidence should produce low confidence, not hallucinated certainty.
9. Reddit/student reviews are noisy signals, not truth.
10. Deterministic operations should stay deterministic.
11. LLMs should only handle genuinely fuzzy reasoning.

12. Avoid using agents where ordinary code is better.
 13. Avoid fake multi-agent debates for resume buzzwords.
 14. Specialized agents + critic + tool use + orchestration is sufficient.
 15. Real users and measurable usage matter more than adding more AI buzzwords.
-

Development Order

Do NOT begin by implementing every agent.

Suggested order:

Stage 1 — Core Product

- authentication
- create semester
- create/add courses
- upload syllabus
- store PDF
- extract course metadata
- correctly extract assessments and dates

Stage 2 — RAG

- chunk syllabus
- embeddings
- pgvector
- syllabus-grounded retrieval
- citations

Stage 3 — External Research

- web search
- Reddit/course research
- evidence extraction
- evidence storage
- source citations

Stage 4 — Evaluation

- workload dimensions
- confidence
- ranking algorithm
- visual dashboard

Stage 5 — Multi-Agent Orchestration

- specialized agents
- course workers
- evaluator
- critic
- revision loop
- fan-out/fan-in processing

Stage 6 — Semester Planning

- combined assessments
- workload heatmap
- study priority
- weekly recommendations

Stage 7 — Production Engineering

- async jobs
- retries
- monitoring
- logging
- analytics
- caching
- rate limiting
- error handling
- security/privacy

Stage 8 — Real Users / Launch

Initially target University of Alberta students.

Get actual students to upload real syllabi and use the system.

Collect feedback and product metrics.

Potential expansion to other Canadian universities afterward.

Long-Term Resume Goal

The ideal resume bullet should eventually look closer to:

"Built and deployed an AI academic planning platform used by X students, combining multi-agent web research, syllabus RAG, PostgreSQL/pgvector, and evidence-grounded workload estimation across X university courses; processed X documents and generated personalized semester workload plans with source-level citations."

NOT:

"Built a multi-agent chatbot using LangChain."

The objective is a real engineered product demonstrating:

- AI engineering
- agent systems
- RAG
- backend engineering
- database design
- asynchronous workflows
- frontend visualization
- cloud deployment
- observability
- real-world users
- product thinking
- measurable impact

When helping with this project, prioritize sound architecture, simplicity, reliability, and real user value over unnecessary AI complexity.